Multi-class Classification

Estimated time needed: **30** mins

In this lab, you will learn the different strategies of Multi-class classification and implement the same on a real-world dataset.

Objectives

After completing this lab you will be able to:

1. Understand the use of one-hot encoding for categorical variables.
2. Implement logistic regression for multi-class classification using **One-vs-All (OvA)** and **One-vs-One (OvO)** strategies.
3. Evaluate model performance using appropriate metrics.

Import Necessary Libraries

First, to ensure the availability of the required libraries, execute the cell below.

```
!pip install numpy==2.2.0
!pip install pandas==2.2.3
!pip install scikit-learn==1.6.0
!pip install matplotlib==3.9.3
!pip install seaborn==0.13.2
```

```
Collecting numpy==2.2.0
  Downloading numpy-2.2.0-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (62 kB)
Downloading numpy-2.2.0-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (16.1 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16.1/16.1 MB 120.9 MB/s eta
0:00:00
Installing collected packages: numpy
Successfully installed numpy-2.2.0
Collecting pandas==2.2.3
  Downloading pandas-2.2.3-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (89 kB)
Requirement already satisfied: numpy>=1.26.0 in
/opt/conda/lib/python3.12/site-packages (from pandas==2.2.3) (2.2.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
/opt/conda/lib/python3.12/site-packages (from pandas==2.2.3)
(2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/opt/conda/lib/python3.12/site-packages (from pandas==2.2.3) (2024.2)
Collecting tzdata>=2022.7 (from pandas==2.2.3)
  Downloading tzdata-2025.3-py2.py3-none-any.whl.metadata (1.4 kB)
```

```
Requirement already satisfied: six>=1.5 in
/opt/conda/lib/python3.12/site-packages (from python-dateutil>=2.8.2-
>pandas==2.2.3) (1.17.0)
Downloading pandas-2.2.3-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (12.7 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.7/12.7 MB 163.8 MB/s eta
0:00:00
anylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (18 kB)
Requirement already satisfied: numpy>=1.19.5 in
/opt/conda/lib/python3.12/site-packages (from scikit-learn==1.6.0)
(2.2.0)
Collecting scipy>=1.6.0 (from scikit-learn==1.6.0)
  Downloading scipy-1.17.1-cp312-cp312-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (62 kB)
Collecting joblib>=1.2.0 (from scikit-learn==1.6.0)
  Downloading joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Collecting threadpoolctl>=3.1.0 (from scikit-learn==1.6.0)
  Downloading threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Downloading scikit_learn-1.6.0-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (13.1 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 13.1/13.1 MB 128.6 MB/s eta
0:00:00
Downloading joblib-1.5.3-py3-none-any.whl (309 kB)
Downloading scipy-1.17.1-cp312-cp312-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (35.2 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 35.2/35.2 MB 189.2 MB/s eta
0:00:00
Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Installing collected packages: threadpoolctl, scipy, joblib, scikit-
learn
Successfully installed joblib-1.5.3 scikit-learn-1.6.0 scipy-1.17.1
threadpoolctl-3.6.0
Collecting matplotlib==3.9.3
  Downloading matplotlib-3.9.3-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (11 kB)
Collecting contourpy>=1.0.1 (from matplotlib==3.9.3)
  Downloading contourpy-1.3.3-cp312-cp312-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib==3.9.3)
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib==3.9.3)
  Downloading fonttools-4.61.1-cp312-cp312-
manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux
_2_5_x86_64.whl.metadata (114 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib==3.9.3)
  Downloading kiwisolver-1.4.9-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (6.3 kB)
Requirement already satisfied: numpy>=1.23 in
/opt/conda/lib/python3.12/site-packages (from matplotlib==3.9.3)
```

```
(2.2.0)
Requirement already satisfied: packaging>=20.0 in
/opt/conda/lib/python3.12/site-packages (from matplotlib==3.9.3)
(24.2)
Collecting pillow>=8 (from matplotlib==3.9.3)
  Downloading pillow-12.1.1-cp312-cp312-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (8.8 kB)
Collecting pyparsing>=2.3.1 (from matplotlib==3.9.3)
  Downloading pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: python-dateutil>=2.7 in
/opt/conda/lib/python3.12/site-packages (from matplotlib==3.9.3)
(2.9.0.post0)
Requirement already satisfied: six>=1.5 in
/opt/conda/lib/python3.12/site-packages (from python-dateutil>=2.7-
>matplotlib==3.9.3) (1.17.0)
Downloading matplotlib-3.9.3-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (8.3 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.3/8.3 MB 115.8 MB/s eta
0:00:00
anylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (362 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.61.1-cp312-cp312-
manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux
_2_5_x86_64.whl (5.0 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.0/5.0 MB 149.6 MB/s eta
0:00:00
anylinux2014_x86_64.manylinux_2_17_x86_64.whl (1.5 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 108.0 MB/s eta
0:00:00
anylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (7.0 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.0/7.0 MB 165.7 MB/s eta
0:00:00
atplotlib
Successfully installed contourpy-1.3.3 cycler-0.12.1 fonttools-4.61.1
kiwisolver-1.4.9 matplotlib-3.9.3 pillow-12.1.1 pyparsing-3.3.2
Collecting seaborn==0.13.2
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in
/opt/conda/lib/python3.12/site-packages (from seaborn==0.13.2) (2.2.0)
Requirement already satisfied: pandas>=1.2 in
/opt/conda/lib/python3.12/site-packages (from seaborn==0.13.2) (2.2.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in
/opt/conda/lib/python3.12/site-packages (from seaborn==0.13.2) (3.9.3)
Requirement already satisfied: contourpy>=1.0.1 in
/opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn==0.13.2) (1.3.3)
Requirement already satisfied: cycler>=0.10 in
/opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn==0.13.2) (0.12.1)
```

```
Requirement already satisfied: fonttools>=4.22.0 in
/opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn==0.13.2) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn==0.13.2) (1.4.9)
Requirement already satisfied: packaging>=20.0 in
/opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn==0.13.2) (24.2)
Requirement already satisfied: pillow>=8 in
/opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn==0.13.2) (12.1.1)
Requirement already satisfied: pyparsing>=2.3.1 in
/opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn==0.13.2) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in
/opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn==0.13.2) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/opt/conda/lib/python3.12/site-packages (from pandas>=1.2-
>seaborn==0.13.2) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in
/opt/conda/lib/python3.12/site-packages (from pandas>=1.2-
>seaborn==0.13.2) (2025.3)
Requirement already satisfied: six>=1.5 in
/opt/conda/lib/python3.12/site-packages (from python-dateutil>=2.7-
>matplotlib!=3.6.1,>=3.4->seaborn==0.13.2) (1.17.0)
Downloading seaborn-0.13.2-py3-none-any.whl (294 kB)
Installing collected packages: seaborn
Successfully installed seaborn-0.13.2
```

Now, import the necessary libraries for data processing, model training, and evaluation.

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsOneClassifier
from sklearn.metrics import accuracy_score

import warnings
warnings.filterwarnings('ignore')
```

About the dataset

The data set being used for this lab is the "Obesity Risk Prediction" data set publically available on UCI Library under the CCA 4.0 license. The data set has 17 attributes in total along with 2,111 samples.

The attributes of the dataset are descibed below.

Load the dataset

Load the data set by executing the code cell below.

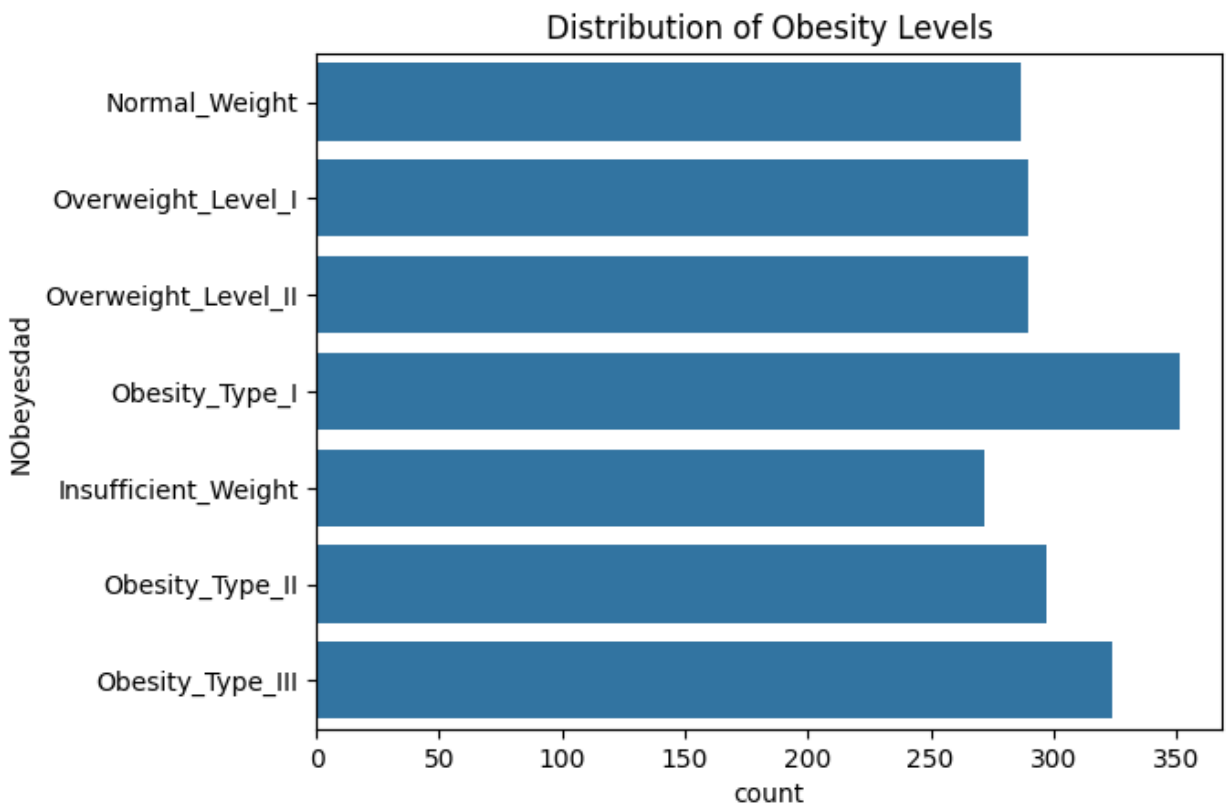
```
file_path = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/GkDzb7bWrtvGXdPOfk6CIg/Obesity-level-prediction-dataset.csv"
data = pd.read_csv(file_path)
data.head()
```

```
   Gender   Age  Height  Weight family_history_with_overweight FAVC
FCVC  \
0  Female  21.0    1.62    64.0                            yes   no
2.0
1  Female  21.0    1.52    56.0                            yes   no
3.0
2    Male  23.0    1.80    77.0                            yes   no
2.0
3    Male  27.0    1.80    87.0                             no   no
3.0
4    Male  22.0    1.78    89.8                             no   no
2.0

   NCP       CAEC SMOKE  CH2O  SCC  FAF  TUE        CALC  \
0  3.0  Sometimes    no   2.0   no  0.0  1.0          no
1  3.0  Sometimes   yes   3.0  yes  3.0  0.0   Sometimes
2  3.0  Sometimes    no   2.0   no  2.0  1.0  Frequently
3  3.0  Sometimes    no   2.0   no  2.0  0.0  Frequently
4  1.0  Sometimes    no   2.0   no  0.0  0.0   Sometimes

                  MTRANS           NObeyesdad
0  Public_Transportation        Normal_Weight
1  Public_Transportation        Normal_Weight
2  Public_Transportation        Normal_Weight
3                Walking   Overweight_Level_I
4  Public_Transportation  Overweight_Level_II
```

Exploratory Data Analysis

Visualize the distribution of the target variable to understand the class balance.

```
# Distribution of target variable
sns.countplot(y='NObeyesdad', data=data)
plt.title('Distribution of Obesity Levels')
plt.show()
```



This shows that the dataset is fairly balanced and does not require any special attention in terms of biased training.

Exercise 1

Check for null values, and display a summary of the dataset (use `.info()` and `.describe()` methods).

```
# Checking for null values
print(data.isnull().sum())

# Dataset summary
print(data.info())
print(data.describe())
```

```
Gender                              0
Age                                 0
Height                              0
Weight                              0
```

```
family_history_with_overweight    0
FAVC                              0
FCVC                              0
NCP                               0
CAEC                              0
SMOKE                             0
CH2O                              0
SCC                               0
FAF                               0
TUE                               0
CALC                              0
MTRANS                            0
NObeyesdad                        0
dtype: int64
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2111 entries, 0 to 2110
Data columns (total 17 columns):
 #   Column                           Non-Null Count  Dtype
---  ------                           --------------  -----
 0   Gender                           2111 non-null   object
 1   Age                              2111 non-null   float64
 2   Height                           2111 non-null   float64
 3   Weight                           2111 non-null   float64
 4   family_history_with_overweight   2111 non-null   object
 5   FAVC                             2111 non-null   object
 6   FCVC                             2111 non-null   float64
 7   NCP                              2111 non-null   float64
 8   CAEC                             2111 non-null   object
 9   SMOKE                            2111 non-null   object
 10  CH2O                             2111 non-null   float64
 11  SCC                              2111 non-null   object
 12  FAF                              2111 non-null   float64
 13  TUE                              2111 non-null   float64
 14  CALC                             2111 non-null   object
 15  MTRANS                           2111 non-null   object
 16  NObeyesdad                       2111 non-null   object
dtypes: float64(8), object(9)
memory usage: 280.5+ KB
None
               Age       Height       Weight         FCVC          NCP
\
count  2111.000000  2111.000000  2111.000000  2111.000000  2111.000000

mean     24.312600     1.701677    86.586058     2.419043     2.685628

std       6.345968     0.093305    26.191172     0.533927     0.778039

min      14.000000     1.450000    39.000000     1.000000     1.000000

25%      19.947192     1.630000    65.473343     2.000000     2.658738
```

```
50%      22.777890     1.700499    83.000000     2.385502     3.000000

75%      26.000000     1.768464   107.430682     3.000000     3.000000

max      61.000000     1.980000   173.000000     3.000000     4.000000


              CH2O          FAF          TUE
count  2111.000000  2111.000000  2111.000000
mean      2.008011     1.010298     0.657866
std       0.612953     0.850592     0.608927
min       1.000000     0.000000     0.000000
25%       1.584812     0.124505     0.000000
50%       2.000000     1.000000     0.625350
75%       2.477420     1.666678     1.000000
max       3.000000     3.000000     2.000000
```

Expected Output:

- Counts of null values for each column (likely zero for this dataset).
- Dataset info including column names, data types, and memory usage.
- Descriptive statistics for numerical columns.

Preprocessing the data

Feature scaling

Scale the numerical features to standardize their ranges for better model performance.

```
# Standardizing continuous numerical features
continuous_columns =
data.select_dtypes(include=['float64']).columns.tolist()

scaler = StandardScaler()
scaled_features = scaler.fit_transform(data[continuous_columns])

# Converting to a DataFrame
scaled_df = pd.DataFrame(scaled_features,
columns=scaler.get_feature_names_out(continuous_columns))

# Combining with the original dataset
scaled_data = pd.concat([data.drop(columns=continuous_columns),
scaled_df], axis=1)
```

Standardization of data is important to better define the decision boundaries between classes by making sure that the feature variations are in similar scales. The data is now ready to be used for training and testing.

One-hot encoding

Convert categorical variables into numerical format using one-hot encoding.

```
# Identifying categorical columns
categorical_columns =
scaled_data.select_dtypes(include=['object']).columns.tolist()
categorical_columns.remove('NObeyesdad')  # Exclude target column

# Applying one-hot encoding
encoder = OneHotEncoder(sparse_output=False, drop='first')
encoded_features =
encoder.fit_transform(scaled_data[categorical_columns])

# Converting to a DataFrame
encoded_df = pd.DataFrame(encoded_features,
columns=encoder.get_feature_names_out(categorical_columns))

# Combining with the original dataset
prepped_data =
pd.concat([scaled_data.drop(columns=categorical_columns), encoded_df],
axis=1)
```

You will observe that all the categorical variables have now been modified to one-hot encoded features. This increases the overall number of fields to 24.

Encode the target variable

```
# Encoding the target variable
prepped_data['NObeyesdad'] =
prepped_data['NObeyesdad'].astype('category').cat.codes
prepped_data.head()
```

```
   NObeyesdad       Age    Height    Weight      FCVC       NCP
CH2O  \
0           1 -0.522124 -0.875589 -0.862558 -0.785019  0.404153 -
0.013073
1           1 -0.522124 -1.947599 -1.168077  1.088342  0.404153
1.618759
2           1 -0.206889  1.054029 -0.366090 -0.785019  0.404153 -
0.013073
3           5  0.423582  1.054029  0.015808  1.088342  0.404153 -
0.013073
4           6 -0.364507  0.839627  0.122740 -0.785019 -2.167023 -
0.013073

        FAF       TUE  Gender_Male  ...  CAEC_no  SMOKE_yes
SCC_yes  \
0 -1.188039  0.561997          0.0  ...      0.0        0.0      0.0
```

```
1  2.339750 -1.080625          0.0  ...      0.0        1.0      1.0

2  1.163820  0.561997          1.0  ...      0.0        0.0      0.0

3  1.163820 -1.080625          1.0  ...      0.0        0.0      0.0

4 -1.188039 -1.080625          1.0  ...      0.0        0.0      0.0


   CALC_Frequently  CALC_Sometimes  CALC_no  MTRANS_Bike
MTRANS_Motorbike  \
0              0.0             0.0      1.0          0.0
0.0
1              0.0             1.0      0.0          0.0
0.0
2              1.0             0.0      0.0          0.0
0.0
3              1.0             0.0      0.0          0.0
0.0
4              0.0             1.0      0.0          0.0
0.0

   MTRANS_Public_Transportation  MTRANS_Walking
0                           1.0             0.0
1                           1.0             0.0
2                           1.0             0.0
3                           0.0             1.0
4                           1.0             0.0

[5 rows x 24 columns]
```

Separate the input and target data

```
# Preparing final dataset
X = prepped_data.drop('NObeyesdad', axis=1)
y = prepped_data['NObeyesdad']
```

Model training and evaluation

Splitting the data set

Split the data into training and testing subsets.

```
# Splitting data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)
```

Logistic Regression with One-vs-All

In the One-vs-All approach:

- The algorithm trains a single binary classifier for each class.
- Each classifier learns to distinguish a single class from all the others combined.
- If there are k classes, k classifiers are trained.
- During prediction, the algorithm evaluates all classifiers on each input, and selects the class with the highest confidence score as the predicted class.

Advantages:

- Simpler and more efficient in terms of the number of classifiers (k)
- Easier to implement for algorithms that naturally provide confidence scores (e.g., logistic regression, SVM).

Disadvantages:

- Classifiers may struggle with class imbalance since each binary classifier must distinguish between one class and the rest.
- Requires the classifier to perform well even with highly imbalanced datasets, as the "all" group typically contains more samples than the "one" class.

Train a logistic regression model using the One-vs-All strategy and evaluate its performance.

```
# Training logistic regression model using One-vs-All (default)
model_ova = LogisticRegression(multi_class='ovr', max_iter=1000)
model_ova.fit(X_train, y_train)
```

```
LogisticRegression(max_iter=1000, multi_class='ovr')
```

You can now evaluate the accuracy of the trained model as a measure of its performance on unseen testing data.

```
# Predictions
y_pred_ova = model_ova.predict(X_test)

# Evaluation metrics for OvA
print("One-vs-All (OvA) Strategy")
print(f"Accuracy: {np.round(100*accuracy_score(y_test, y_pred_ova),2)}
%")
```

```
One-vs-All (OvA) Strategy
Accuracy: 76.12%
```

Logistic Regression with OvO

In the One-vs-One approach:

- The algorithm trains a binary classifier for every pair of classes in the dataset.
- If there are k classes, this results in $k(k-1)/2$ classifiers.

- Each classifier is trained to distinguish between two specific classes, ignoring the rest.
- During prediction, all classifiers are used, and a "voting" mechanism decides the final class by selecting the class that wins the majority of pairwise comparisons.

Advantages:

- Suitable for algorithms that are computationally expensive to train on many samples because each binary classifier deals with a smaller dataset (only samples from two classes).
- Can be more accurate in some cases since classifiers focus on distinguishing between two specific classes at a time.

Disadvantages:

- Computationally expensive for datasets with a large number of classes due to the large number of classifiers required.
- May lead to ambiguous predictions if voting results in a tie.

Train a logistic regression model using the One-vs-One (OvO) strategy and evaluate its performance.

```
# Training logistic regression model using One-vs-One
model_ovo = OneVsOneClassifier(LogisticRegression(max_iter=1000))
model_ovo.fit(X_train, y_train)
```

```
OneVsOneClassifier(estimator=LogisticRegression(max_iter=1000))
```

Evaluate the accuracy of the trained model as a measure of its performance on unseen testing data.

```
# Predictions
y_pred_ovo = model_ovo.predict(X_test)

# Evaluation metrics for OvO
print("One-vs-One (OvO) Strategy")
print(f"Accuracy: {np.round(100*accuracy_score(y_test, y_pred_ovo),2)}
%")
```

```
One-vs-One (OvO) Strategy
Accuracy: 92.2%
```

Exercises

Q1. Experiment with different test sizes in the train_test_split method (e.g., 0.1, 0.3) and observe the impact on model performance.

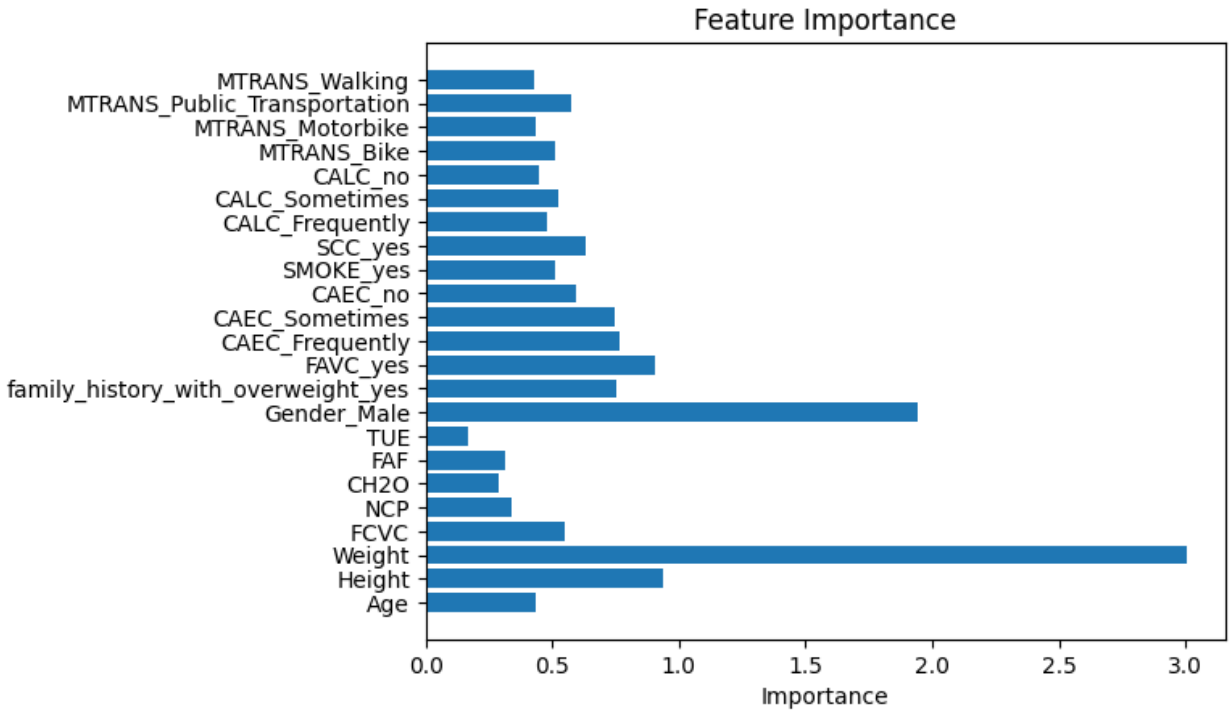
```
# your code here
for test_size in [0.1, 0.3]:
    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=test_size, random_state=42, stratify=y)
    model_ova.fit(X_train, y_train)
```

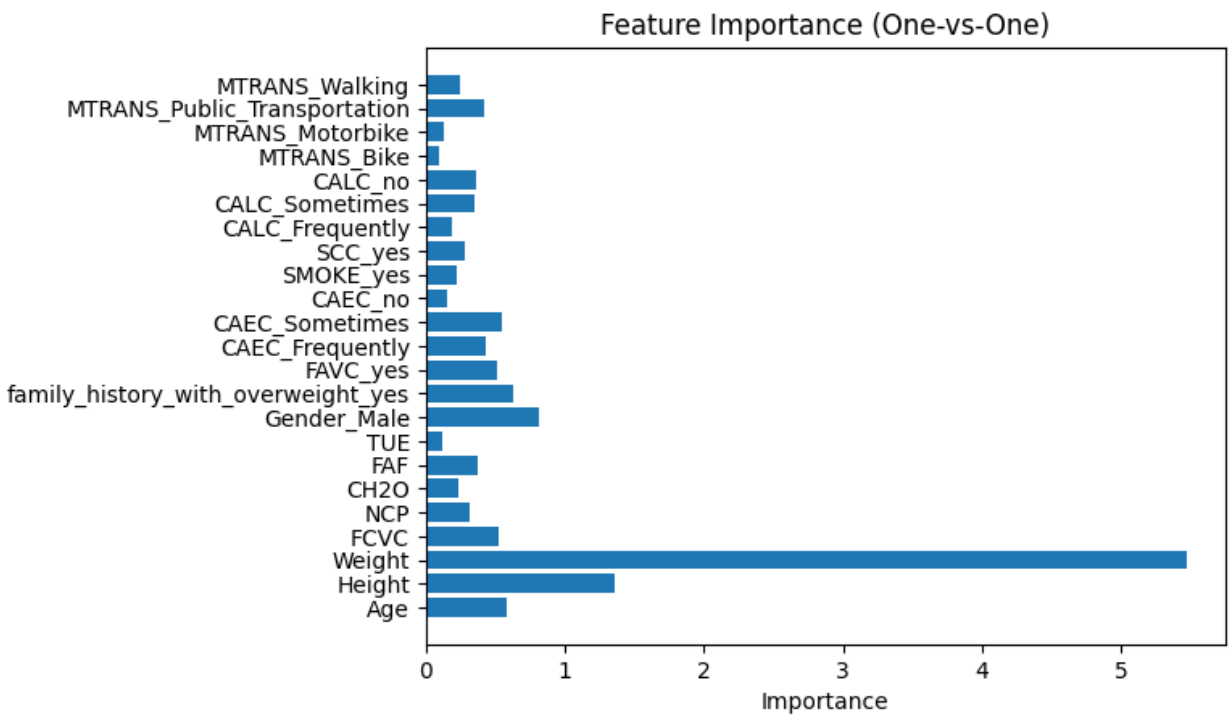
```
    y_pred = model_ova.predict(X_test)
    print(f"Test Size: {test_size}")
    print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Test Size: 0.1
Accuracy: 0.7594339622641509
Test Size: 0.3
Accuracy: 0.749211356466877
```

Q2. Plot a bar chart of feature importance using the coefficients from the One vs All logistic regression model. Also try for the One vs One model.

```
# your code here
feature_importance = np.mean(np.abs(model_ova.coef_), axis=0)
plt.barh(X.columns, feature_importance)
plt.title("Feature Importance")
plt.xlabel("Importance")
plt.show()

# For One vs One model
# Collect all coefficients from each underlying binary classifier
coefs = np.array([est.coef_[0] for est in model_ovo.estimators_])

# Now take the mean across all those classifiers
feature_importance = np.mean(np.abs(coefs), axis=0)

# Plot feature importance
plt.barh(X.columns, feature_importance)
plt.title("Feature Importance (One-vs-One)")
plt.xlabel("Importance")
plt.show()
```





Q3. Write a function `obesity_risk_pipeline` to automate the entire pipeline: Loading and preprocessing the data Training the model Evaluating the model The function should accept the file path and test set size as the input arguments.

```
# write your function here and then execute this cell
def obesity_risk_pipeline(data_path, test_size=0.2):
# your code here

obesity_risk_pipeline(file_path, test_size=0.2)
```

Congratulations! You're ready to move on to your next lesson!

Author

Abishek Gagneja

Other Contributors

Jeff Grossman

<!--

Changelog

| Date | Version | Changed by | Change Description |

|:----------|:------|:----------|:----------|

| 2024-11-05 | 1.0 Abhishek Gagnejan | Fresh version created | | 2025-05-13 | 1.1 Anita Verma | Added the solution code for Ovo model Q2 |

© IBM Corporation. All rights reserved.